

Joint Lung + Nodule Segmentation — 3D U-Net / DynUNet (ex-LIDC)

1. Task

Voxel-level joint segmentation of lung tissue and pulmonary nodules in 3D chest CT volumes. Single model, single forward pass, no ROI stage and no bounding-box crop.

- **Input** (1, 256, 256, 256) full CT resampled to 256^3 , intensity-normalised to [0, 1].
- **Output** (3, 256, 256, 256) softmax logits: class 0 = background, class 1 = lung, class 2 = nodule.

A voxel that is both lung tissue and nodule is assigned exclusively to class 2 (nodule takes precedence over lung) since the head uses a mutually-exclusive softmax.

2. Model

[MONAI DynUNet](#) — nnU-Net-style dynamic **3D U-Net**, DynUNet (nnU-Net-style 3D U-Net).

Property	Value
Architecture family	DynUNet (nnU-Net-style 3D U-Net)
Spatial dims	3
Input channels	1
Output channels	3 (softmax)
Encoder levels	6
Kernel sizes	[3, 3, 3] at every level
Strides	[[1,1,1], [2,2,2], [2,2,2], [2,2,2], [2,2,2], [2,2,2]]
Normalisation	instance
Deep supervision	off
Trainable parameters	31,181,763

3. Data

Trained on the `unified_ex_lidc` split (patient-grouped, dataset-stratified, **LIDC excluded**):

Source	Role
NLST	train + val + test

Source	Role
NSCLC-Radiomics	train + val + test
~~LIDC-IDRI~~	<i>excluded</i>

Split sizes: **1 110 train / 196 val / 129 test (held out)**.

Lung labels are ground-truth per-slice 2D masks (`roi_sem_seg_2d/`), stacked into 3D per series. Nodule labels come from the corpus's own 3D nodule annotations.

4. Training

Setting	Value
Loss	Multi-class Focal Tversky + weighted CE ($\alpha=0.3$, $\beta=0.7$, $\gamma=2.0$, $\lambda_{CE}=0.1$, class weights = <code>[1.0, 1.0, 100.0]</code> for <code>[bg, lung, nodule]</code>)
Optimizer	Adam
Learning rate	1×10^{-5}
Weight decay	1×10^{-5}
LR schedule	Cosine annealing, $T_{max} = 400$, $\eta_{min} = 1 \times 10^{-6}$
Batch size	2
Epochs	400
Random seed	42
Mixed precision	bf16 (autocast, no GradScaler)
Augmentation	3D flips, 90° rotations, elastic rotation, zoom, intensity scale/shift, Gaussian noise/blur, contrast
Hardware	1 × NVIDIA H100 94 GB
Wall-clock	≈ 3.5 days

Best checkpoint (evaluated below): **epoch 159 / 400**.

5. Evaluation protocol

Held-out test split (129 series). The training loop used the `val` split for early stopping and best-Dice checkpoint selection; the numbers below come from the completely untouched `test` split (never seen during training or model selection). Predictions taken as `argmax` over the 3-channel softmax output. All per-voxel metrics are micro-averaged over the whole test split (2,164,260,864 voxels total).

6. Results

Ticket-required metrics

Metric	Value
mean IoU (all 3 classes)	0.8016
mean IoU (lung + nodule)	0.7046
Accuracy	0.9957
Precision (macro, all classes)	0.8244
Precision (macro, non-bg)	0.7375
Recall (macro, all classes)	0.9354
Recall (macro, non-bg)	0.9046

Per-class breakdown

Class	IoU	Precision	Recall	Dice (micro)
Background	0.9955	0.9983	0.9971	0.9977
Lung	0.9535	0.9718	0.9807	0.9762
Nodule	0.4557	0.5032	0.8285	0.6261

Per-case Dice distribution (per class, 129 val cases)

Class	Mean	Std	p25	Median	p75
Background	0.9977	0.0009	0.9969	0.9980	0.9985
Lung	0.9688	0.0217	0.9651	0.9717	0.9806
Nodule	0.5489	0.2665	0.3691	0.6235	0.7758

7. Notes on interpretation

The joint task's class distribution is heavily skewed: background dominates, lung tissue is ~14 % of the volume, and nodule voxels are on the order of 10^{-5} . This shapes what each metric means here:

- **Accuracy** looks near-perfect (~ 0.99) because getting background right accounts for most voxels regardless of model quality. Read it as a floor, not a discriminator.
- **mIoU** across all three classes is dominated by the near-1.0 `IoU_background`; the *no-bg* variant (`IoU_lung`, `IoU_nodule` averaged) is the more diagnostic number for comparing the two model families.
- **Precision / Recall** for the lung head sit at ~0.98 — the model has learned lung anatomy well. The nodule head has Precision and Recall in the 0.62–0.75 range and is where architecture / data choices actually differentiate.

- **Per-case Dice** for the nodule class carries a wide distribution ($\text{std} \approx 0.25$) driven by nodule size and location — the median is a better summary than the mean for clinical use.

8. How to load & run inference

The pre-trained weights are hosted at [szabopeter/joint-dynunet-3d-ex-lidc](https://huggingface.co/szabopeter/joint-dynunet-3d-ex-lidc). Download and load directly:

```
huggingface-cli download szabopeter/joint-dynunet-3d-ex-lidc --local-dir ./ckpt
```

```
import yaml, torch
from monai.networks.nets import DynUNet

cfg = yaml.safe_load(open("config.yaml"))["model"]
default_strides = [[1, 1, 1]] + [[2, 2, 2]] * 5
strides = cfg.get("strides", default_strides)
kernel = cfg.get("kernel_size", [[3, 3, 3]] * len(strides))
model = DynUNet(
    spatial_dims      = cfg["spatial_dims"],
    in_channels       = cfg["in_channels"],
    out_channels      = cfg["out_channels"],
    kernel_size       = kernel,
    strides           = strides,
    upsample_kernel_size = cfg.get("upsample_kernel_size", strides[1:]),
    norm_name         = cfg.get("norm_name", "instance"),
    deep_supervision  = cfg.get("deep_supervision", False),
)
state = torch.load("model.pth", map_location="cpu", weights_only=True)
model.load_state_dict(state)
model.eval()

with torch.no_grad():
    x = torch.randn(1, 1, 256, 256, 256)          # full CT resampled to 2563
    logits = model(x)                             # (B, 3, D, H, W)
    pred_class = logits.argmax(dim=1)              # (B, D, H, W) in {0, 1, 2}
    lung_mask = (pred_class == 1).to(torch.uint8)
    nodule_mask = (pred_class == 2).to(torch.uint8)
```

Unlike the two-stage `nodule-*` HF checkpoints, this model does **not** need a lung-bbox crop — feed it the whole CT resampled to 256³.

Instance-level metrics (per-nodule)

Connected-component analysis with 26-connectivity, pooled over the held-out test split, on the 256-cube resample grid (same grid as the voxel metrics above).

- **Instance recall** — GT nodules touched by at least one predicted voxel, over all GT nodules.

- **Instance precision** — predicted components touching at least one GT voxel, over all predicted components.

Metric	Value
GT nodules (components)	258
GT nodules hit	161
Instance recall	0.6240
Predicted components	550
Predicted components hitting GT	427
Instance precision	0.7764

No minimum-size filtering is applied — every predicted component counts, so single-voxel false positives lower instance precision. Produced by `scripts/instance_metrics.py --config configs/joint_dynunet_ex_lidc.yaml --ckpt <ckpt> --split test` (JSON: `results_instance/joint_dynunet_ex_lidc.json`).

9. Reproducibility

- Config `configs/joint_dynunet_ex_lidc.yaml`
- Checkpoint `checkpoints/joint_dynunet_ex_lidc/best_model.pth`
- Metrics JSON `checkpoints/joint_dynunet_ex_lidc/eval_metrics_joint_test.json`
- Command `python eval_metrics_joint.py --config configs/joint_dynunet_ex_lidc.yaml --ckpt checkpoints/joint_dynunet_ex_lidc/best_model.pth --split test`

Companion variant: [szabopeter/joint-dynunet-3d-pseudo-lidc](#) — same architecture, pseudo-LIDC data variant.